SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

COSTSCALE

ML-Driven Hybrid Container and Serverless Resource Management on AWS

Detailed Project Report

Submitted by

Anamay Narkar - 16010123040

Ananya Dhiman - 16010123041

Anoushka Nelson - 16010123049

B.Tech Computer Engineering, Final Year

K.J. Somaiya School of Engineering, Mumbai

[TO FILL: course name] (cloud computing project)

October 2026

Repository: [TO FILL]

Abstract

Cloud providers sell computation both as cheap, slow-to-provision capacity (virtual machines and containers) and as expensive, instant, pay-per-use functions. Plain reactive autoscaling follows load with a delay and, without a fallback, drops or delays the load that arrives while capacity catches up. CostScale combines the two: a base of always-on EC2 capacity sized from an hourly quantile forecast of load, with everything above the base sent to AWS Lambda through an Application Load Balancer with weighted target groups.

The base size follows from the newsvendor problem: it is the q*-quantile of forecast load, with q* = 1 - f/L, where f is the container cost and L the Lambda cost of serving one unit of load for a minute. A trace-driven simulation over 101 heavy functions of the Azure Functions 2019 trace, priced with AWS ap-south-1 prices and the container capacity measured on AWS, finds that a hybrid with a rolling quantile base costs $32.2k per month against $54.6k for all-Lambda and $37.5k for the cheapest reactive setting, which also drops 9.2% of the load; the hybrids drop none. A LightGBM quantile forecaster is 0.25% cheaper than the rolling baseline on test, which is within noise, so most of the benefit comes from the newsvendor rule rather than from machine learning.

A live AWS deployment in ap-south-1 ran four time-compressed experiment arms against the same trace window and compared them with a simulator replay of each arm. On a smooth six-hour ramp, reactive scaling (arm A) and the ML hybrid (arm C) cost the same ($0.038 and $0.039) with p95 latencies of 566 ms and 585 ms, while a static base (arm B) cost 2.2 times as much. With an unseen 4x spike (arm D) no request failed and 48% of that hour's requests went to Lambda, but 20% of all requests exceeded the 800 ms target because the load balancer let EC2 run at its limit. The live runs therefore show that the mechanism works and expose its weak points, but do not show an advantage over reactive scaling on this window; the cost advantage appears in the simulation across bursty functions. Energy is only an estimate; storage and networking are out of scope.

Keywords: cloud resource management, workload prediction, quantile forecasting, newsvendor, AWS Lambda, EC2 Auto Scaling, Application Load Balancer, LightGBM, serverless

Table of contents

1. Introduction **7**
1.1 Background 7
1.2 Idea in one paragraph 7
1.3 Contributions 7
1.4 Scope 8
2. Problem statement **9**
2.1 What the system must decide 9
3. Related work **10**
4. Data **12**
4.1 Dataset 12
4.2 Filtering and joins 12
4.3 Load model and split 13
4.4 Heavy functions 13
5. Methodology **14**
5.1 Cost model 14
5.2 Newsvendor rule for the base 14
5.3 Forecaster 14
5.4 Strategies compared 15
6. System design **16**
6.1 Architecture 16
6.2 Two time scales 16
6.3 Controller algorithm 16
7. Implementation **18**
7.1 Repository and tooling 18
7.2 Application 18
7.3 Workload for the live demo 18
7.4 Calibration 19
7.5 What was not run live 19
8. Evaluation **20**
8.1 Set-up 20
8.2 Prediction accuracy 20
8.3 Decision cost, utilisation and unserved load 22
8.4 Does the ML forecaster help? 24
8.5 Latency 24
8.6 Energy (estimate) 25
8.7 Per-function distribution and sensitivity 25
8.8 Live results and sim-to-real gap 26
8.9 Validation of the newsvendor rule 30
9. What happens when the prediction is wrong **31**

10. Limitations..... 33
11. Conclusion..... 35
12. Future work..... 36
References..... 37
Appendix A: Reproduction steps..... 39
A.1 Parameters..... 39

List of figures

Figure 1: CostScale architecture
Figure 2: Observed coverage against nominal quantile on test days
Figure 3: Hourly base capacity for the live-demo function: forecasts against the realised hourly quantile
Figure 4: Monthly cost by strategy; hatching marks strategies that drop load
Figure 5: Cost against unserved load for the reactive and seasonal sweeps, and the hybrids
Figure 6: Hybrid-rolling cost relative to all-Lambda under each scenario
Figure 7: Live timelines: request rate, base capacity and EC2 share for each arm
Figure 8: Live p95 latency per window and cumulative cost for each arm
Figure 9: Effect of an unseen spike on cost and unserved load

List of tables

Table 1: Decisions the resource manager makes
Table 2: Dataset funnel
Table 3: Strategies in the simulator
Table 4: Repository layout
Table 5: Calibration results
Table 6: Pinball loss on test (normalised; lower is better)
Table 7: Coverage on test (ideal equals q)
Table 8: Mean absolute error of the forecast against the realised hourly quantile (test, normalised)
Table 9: Test results, EC2 base (measured capacity)
Table 10: Decision cost of the hybrid base policies
Table 11: Sensitivity of cost to assumptions
Table 12: Quantile-level sweep for hybrid-rolling on validation (offset 0 = theoretical q*)
Table 13: Live results: latency, errors and routing
Table 14: Live results: backend latency, resources, cost and energy estimate
Table 15: Reconciliation: live arms against a simulator replay of the same arm
Table 16: Simulated spike results (monthly cost, test)
Table 17: Key parameters

1. Introduction

1.1 Background

Cloud providers sell the same computation in two very different ways. Containers and virtual machines are cheap per unit of work but must be provisioned ahead of demand: a new EC2 instance takes tens of seconds to minutes to become useful. Function-as-a-Service platforms such as AWS Lambda react instantly and bill only for the milliseconds used, but each unit of work costs several times more. Production traffic is rarely flat, so an operator who sizes capacity for the peak pays for idle machines, and an operator who relies on pure reactive scaling suffers while capacity catches up with the load.

Workload studies of a large public FaaS platform show why this matters. Shahrad et al. characterise the full Azure Functions workload and report that invocation rates vary across roughly eight orders of magnitude and that most functions are invoked rarely [1]. A small number of busy functions therefore dominate cost, and these are exactly the functions where the choice between a reserved base and pay-per-use matters.

1.2 Idea in one paragraph

CostScale keeps a **base** of cheap always-on capacity (an EC2 Auto Scaling Group) sized from an hourly **quantile forecast** of load, and sends everything above the base to AWS Lambda. An Application Load Balancer with two weighted target groups splits traffic between the two. Forecasting decides the slow thing (how many instances to run, which needs lead time); a fast per-minute loop only moves routing weights. The size of the base is not a guess: it follows from the newsvendor problem, so the optimal base is a specific quantile of the forecast load whose level depends only on the price ratio between the two platforms.

1.3 Contributions

- A cost model and a closed-form rule for the base size (Section 5.2), checked against brute-force search (Section 8.9).
- A trace-driven simulator that replays the Azure Functions 2019 trace [8] with AWS ap-south-1 prices over 101 heavy functions, comparing eight strategies on cost, unserved load, utilisation and an energy estimate.
- A global LightGBM quantile forecaster [7] with honest baselines, including a plain statement of how much it does and does not help (Section 8.2).
- A live AWS implementation with a calibration campaign and four experiment arms, reconciled against a simulator replay of the same arms (Section 8.8).

1.4 Scope

The project covers compute only. Storage and networking costs are out of scope because the public trace contains no usable data for them; they appear as future work. Azure traffic is replayed against **AWS prices**: the Azure trace supplies the shape and size of demand, AWS supplies the price list and the platform on which the live system runs. This is a modelling choice and is stated wherever cost numbers appear.

2. Problem statement

Given a stream of requests whose rate varies over time, choose at every moment how much capacity to hold in cheap-but-slow-to-provision form (the base) and how much load to send to expensive-but-instant form (overflow), so that total cost is minimised while every request is served within a latency target. Two kinds of error are possible, and they are not symmetric: a base that is too large wastes money continuously, while a base that is too small sends extra load to the costlier platform but does not drop it.

2.1 What the system must decide

Table 1: Decisions the resource manager makes

Question	CostScale answer	Where
Input data and features	Per-minute invocations, average duration and memory of Azure Functions; last-hour and last-6-hour statistics, same hour yesterday, hour of day, trigger, app level	Sections 4, 5.3
ML algorithm	Global LightGBM quantile regression, one model per quantile	Section 5.3
Prediction horizon	Next hour, decided at the start of each hour	Section 5.3
Allocation mechanism	ASG desired capacity from forecast (hourly); ALB weights from observed rate (per minute)	Section 6
Evaluation	Accuracy, utilisation, latency, cost, energy estimate	Section 8
Wrong prediction	Spike injection in simulation and a live arm; asymmetric error discussion	Section 9

3. Related work

Only sources that were opened and read while preparing this report are cited.

Serverless workloads. Shahrad et al. [1] analyse the production Azure Functions workload, find extreme variation in invocation rates, and propose a policy that reduces cold starts while using fewer resources than common industry approaches. Their trace is the data source for this work [8]. Their focus is the keep-alive and pre-warming of functions inside a serverless platform, whereas CostScale asks the inverse question: when is it cheaper to leave serverless for provisioned capacity, and how much?

Predictive autoscaling with serverless overflow. MArk [2] is a machine-learning inference serving system built on AWS. According to the paper's abstract it uses predictive autoscaling to hide provisioning latency at low cost, and uses more flexible but pricier serverless instances to absorb occasional spikes that are hard to predict.

Hybrid VM and serverless designs. Several works combine provisioned VMs with serverless functions, and some are closer to CostScale than MArk. LIBRA [10] (IEEE IC2E 2021) provisions both VM and serverless capacity to minimise cost under SLAs; its abstract reports more than 85% fewer SLA violations and up to 53% lower cost on real traces, in simulation and on AWS, and does not give the routing algorithm. Spandana [11] (arXiv preprint, 2026) places a lightweight controller next to each application VM that decides **per request** whether to serve it locally or forward it to a FaaS layer such as AWS Lambda, keeping requests that can meet the SLO on the VM; its abstract reports 76-86% CPU utilisation and 5-44% lower cost than three baselines. Flare [12] (arXiv preprint, 2026) runs steady traffic on VMs and shifts the excess load of overloaded microservices to serverless. We read only the abstracts of [10]-[12], not the full papers. CostScale therefore does not claim that a VM base with serverless overflow is new. What it adds, as far as the abstracts show, is an explicit price-ratio rule for the size of the base (the newsvendor quantile of Section 5.2) and a measured comparison with reactive scaling on a public production trace plus a live AWS check. Unlike Spandana, it routes by weighted target groups rather than per request, which is the weaker mechanism (Section 8.8).

Provider predictive scaling. Amazon EC2 Auto Scaling offers predictive scaling [3]. The documentation states that the monitored metric needs at least 24 hours of data, that the service analyses up to 14 days of history, generates an hourly forecast of capacity for the next 48 hours updated every 6 hours, and by default scales at the start of each hour. It forecasts capacity from a utilisation target and does not consider a pay-per-use overflow tier. We did not run it live

because it needs 24 hours of history before it forecasts (Section 7.5); the simulator contains an emulation in the same spirit, which is explicitly **not** the AWS algorithm.

Newsvendor theory. The newsvendor model chooses an order quantity before demand is known. In critical-fractile form the optimum is a quantile of the demand distribution whose level balances the cost of too little against the cost of too much [6]. We apply it per minute of load: the base is bought in advance, the overflow is bought on demand at a higher price.

Forecasting and energy. LightGBM [7] is a gradient-boosted decision tree implementation; its quantile objective gives one conditional quantile per model. For the energy estimate we follow the Cloud Carbon Footprint methodology [9], which converts vCPU-hours and utilisation into watt-hours and treats Lambda through allocated memory.

4. Data

4.1 Dataset

We use the Azure Functions 2019 trace [8], released under CC-BY, which accompanies [1]. It contains 14 days of per-minute invocation counts per function with trigger type, per-function execution-time statistics (excluding cold starts) and per-application allocated memory for 12 of the 14 days. The trace covers 72,359 distinct functions in total; day 1 alone has 46,412, and 28,859 appear on all 14 days. The median function receives only 32 invocations per day on day 1, so most of the data is unusable for forecasting and must be filtered.

4.2 Filtering and joins

Loading all 14 days of all functions at once exhausts memory on the development machine, so the pipeline works in two passes: the first streams every file and computes per-function totals; the second loads only the selected functions as 32-bit integers. A function is selected when it is present on all 14 days (required for a chronological split) and averages at least 5 requests per minute over the 14 days. This gives 2,127 functions; 2,066 also have duration and memory data. Duration is the Count-weighted mean over days and memory is the SampleCount-weighted mean per application.

One difference from the preliminary exploration is documented here for transparency. Applying the 5 requests per minute rule alone gives 2,594 functions; the 2,127 and 2,066 figures are reproduced exactly only when presence on all 14 days is also required, so that is the rule used.

Table 2: Dataset funnel

Quantity	Value
Distinct functions in 14 days	72,359
Functions on day 1	46,412
Functions present on all 14 days	28,859
Median invocations/day (day 1)	32
Average >= 5 req/min, present all 14 days	2,127
... with traffic + duration + memory	2,066
Heavy functions (Section 4.4, measured capacity)	101
Heavy by trigger	event 20, http 16, orchestration 8, queue 52, storage 5

4.3 Load model and split

Offered load is expressed as concurrency: c(t) = invocations per minute / 60 x average duration in seconds. Timer-triggered functions are periodic rather than diurnal and are excluded from forecasting. The split is chronological per function: days 1-10 train, days 11-12 validation, days 13-14 test. All thresholds and hyperparameters are chosen on validation only, and the test period is evaluated once at the end.

4.4 Heavy functions

A function is heavy when the cheaper of its two single-platform options costs at least $50 per month on the **training** days: min(all-Lambda, containers sized to the training peak x 1.2). Using only training days avoids selection leakage. With the measured container capacity this yields 101 functions, all non-timer, and these form the simulated population (101 functions). The preliminary exploration, run with the assumed capacity, found 96; the count depends on the capacity per container, and the earlier assumed-capacity run of this pipeline gave 94.

5. Methodology

5.1 Cost model

Prices were retrieved from the AWS Price List API for ap-south-1 on 2026-10-24: Lambda $0.0000167 per GB-second (tier 1) and $0.0000002 per request; EC2 t3.medium $0.0448 per hour; Fargate $0.04256 per vCPU-hour and $0.004655 per GB-hour. **Azure traffic is priced with AWS prices.** Lambda memory is max(128 MB, application memory). The Lambda cost of one concurrent second is GB x price per GB-second + price per request / average duration in seconds.

Container capacity: the measured capacity of one t3.medium is 16 requests per second at the latency target (Section 7.4), which at the live service time of 0.4 s is 6.4 concurrent requests. The initial assumption was 16, so the simulator was re-run with the measured value; the Fargate figure (4 concurrent requests per 0.5 vCPU / 1 GB task) is still the assumed one because Fargate was not run live. Container cost per capacity-unit-minute is f; Lambda cost to serve one unit of load for one minute is L.

5.2 Newsvendor rule for the base

In every minute the load c is random. A base of b capacity units costs f per unit-minute whether used or not; load above b is served by Lambda at L per unit-minute. The expected cost f b + L E[(c - b)+] is convex in b. Setting its derivative to zero gives f - L P(c > b) = 0, so the optimal base is **b* = F^-1(q*) with q* = 1 - f / L**, the q*-quantile of the load distribution. If L <= f the base is zero, because containers are never cheaper than overflow. The cost ratio therefore fixes the quantile and the forecaster only needs to supply that quantile of the next hour. This is the critical-fractile result of the newsvendor model [6].

With the measured capacity the median q* across the simulated functions is 0.49 and no function has q* = 0. A direct sweep of the quantile level on validation (Table 12) shows that the theoretical level is the cheapest of the levels tried.

5.3 Forecaster

- **Target.** At the start of each hour, for each function, the q-quantile of per-minute concurrency over the next hour, for q in {0.5, 0.7, 0.8, 0.9, 0.95, 0.99}. The base uses q* by linear interpolation between these anchors (with 0 anchored at 0).
- **Model.** One global LightGBM model per quantile [7], objective quantile, trained across all simulated functions with load normalised by each function's training-period mean.

Trigger type is a categorical feature. The model trains on ten sampled minutes per hour and sees training days 2-10.

- **Features** (all from minutes before the decision time): mean, maximum and 90th percentile of the last 60 and last 360 minutes; mean and 90th percentile of the same hour yesterday; sine and cosine of hour of day; trigger type; log of the function's mean level.
- **Safety margin.** A multiplier on the base, chosen on validation by minimising decision cost rather than error. The chosen value is 1.0, i.e. no extra margin.
- **Baselines.** Persistence (quantile of the last hour), rolling 6 hours (quantile of the previous 360 minutes), seasonal naive (same hour yesterday), and a static quantile of the training period.

5.4 Strategies compared

Table 3: Strategies in the simulator

#	Strategy	Definition
1	All-Lambda	Everything on Lambda
2	Peak-sized	Containers sized to training peak x 1.2; no fallback
3	Reactive	Capacity = load seen 5 min ago / target utilisation u; u in {0.7, 0.5, 0.3, 0.2, 0.1}; no fallback
4	Seasonal (emulation)	Capacity from the same hour on the previous 3 days / u. An emulation in the style of predictive scaling, not the AWS algorithm
5	Hybrid-static	Base = q* quantile of training load, constant
6	Hybrid-rolling	Base = q* quantile of previous 6 h, reset hourly
7	Hybrid-ML	Base = LightGBM forecast at q*
8	Hybrid-oracle	Base = q* quantile of the actual hour (lower bound)

In a hybrid, **overflow** is load above base capacity and is billed on Lambda. **Unserved load** is load above container capacity with no Lambda fallback; it is non-zero only for strategies 2, 3 and 4. Each strategy is priced with an EC2 base and, as an alternative, with a Fargate base.

6. System design

6.1 Architecture

Clients send HTTP requests to an internet-facing Application Load Balancer. Its listener forwards to two target groups with weights between 0 and 999: requests are distributed in proportion to the weights [5]. tg-ec2 contains an Auto Scaling Group of t3.medium instances (the base); tg-lambda contains one Lambda function (the overflow). A local controller reads forecasts and observed load and drives both the ASG and the weights; custom metrics go to CloudWatch (Figure 1).

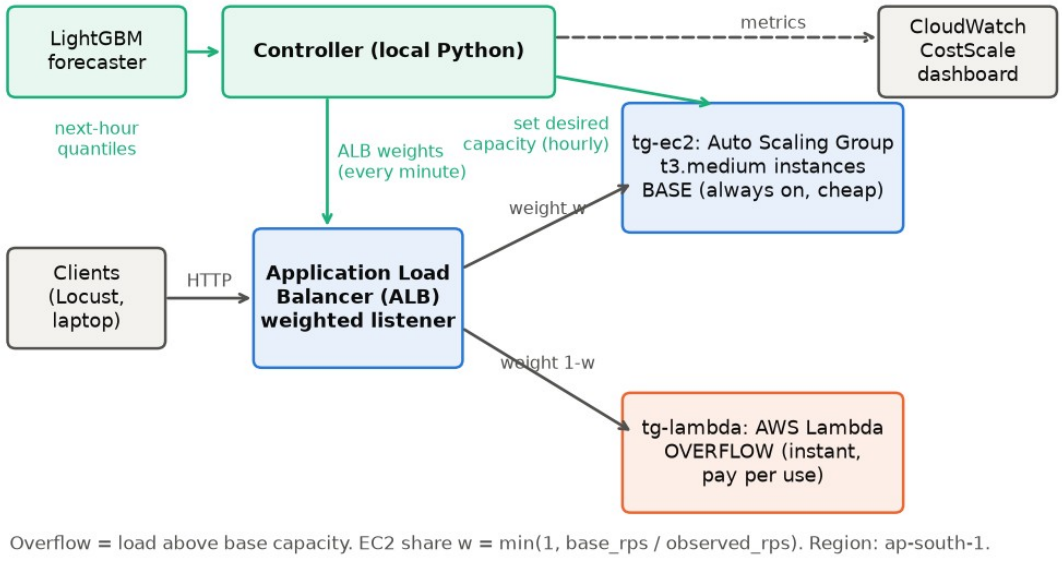


Figure 1: CostScale architecture

6.2 Two time scales

Instances need lead time, so instance count is set from a forecast, once per decision interval. Routing weights are cheap to change, so they are recomputed every minute from the observed request rate: the EC2 share is w = min(1, base capacity / observed rate), with integer weights 0-999 and the Lambda weight equal to 999 minus the EC2 weight. The base capacity used in this formula is the capacity of instances that are healthy in the target group, so traffic is never sent to an instance that is still booting.

6.3 Controller algorithm

```
every decision interval (start of trace hour h):
    Q <- forecast quantiles of next hour (LightGBM) ; q* <- 1 - f/L
    base_rps <- interp(q*, Q) ; n <- ceil(base_rps / cap_per_instance)
    ASG.set_desired_capacity(n)
every tick (one trace minute):
    obs <- observed request rate ; cap <- healthy_instances * cap_per_instance
    w_ec2 <- round(999 * min(1, cap / obs)) ; w_lambda <- 999 - w_ec2
```

```
    ALB.modify_listener(weights) ; publish Forecast, Actual, BaseCapacity,
Weight, RunningCost
```

The ALB splits by percentage and cannot send only the overflow to Lambda, so with random arrivals some requests go to EC2 when EC2 is already full and some go to Lambda when EC2 has room. The live system therefore leaks relative to the simulator; the size of this sim-to-real gap is measured in Section 8.8.

7. Implementation

7.1 Repository and tooling

Python 3 with a virtual environment; boto3 for AWS (OpenTofu was not installed and installing it was declined, so infrastructure is created and destroyed by `infra/up.py` and `infra/down.py`). Every AWS resource is tagged `project=costscale`; the default VPC is used; there is no NAT gateway and no RDS. AWS Budgets alerts are set at $30, $60 and $80. By our own estimate the whole live phase cost roughly half a dollar (about 5 instance-hours, about 3.4 hours of load balancer and under $0.1 of Lambda); AWS's billing figure lags by about a day and was not available when this report was written.

Table 4: Repository layout

Directory	Content
data/	two-pass pipeline (`pipeline.py`), processed arrays (not committed)
sim/	cost model, strategies, sensitivity, newsvendor check
forecast/	features and LightGBM training, baselines, spike test
infra/	boto3 create/destroy, tag verification, spend report
app/	HTTP service and Lambda handler sharing one work module
loadgen/	calibration, workload selection, Locust file
controller/	forecast-driven controller, experiment runner, analysis
results/, report/, ppt/, demo/	outputs and deliverables

7.2 Application

The service exposes `/work?ms=N&cpu=C`: it burns C milliseconds of calibrated single-core CPU and waits for the rest of N milliseconds, and returns the name of the backend that served it. The EC2 version is a pre-forking standard-library HTTP server with one worker per vCPU, started by user-data; the Lambda version is the same module behind an ALB handler, configured with 1769 MB. At that size a function has the equivalent of one vCPU [4]. The live request uses N = 400 ms with C = 80 ms. This is a fixed service time chosen for cost, not the trace's own average duration; the trace is replayed in rate only.

7.3 Workload for the live demo

One heavy HTTP function with a clear daily pattern and moderate burstiness was chosen by score: the highest day-to-day correlation of its hourly profile among the heavy HTTP functions

(r = 0.91) with a minute-level coefficient of variation between 0.1 and 0.6 (here 0.21). A six-hour window on day 14 with the largest rise in load was selected; the rate was scaled by a factor of 6.83 so that the peak is 28 requests per second. The window is time-compressed so that one trace hour lasts 375 seconds, which is four times the longest measured instance launch time (93.8 s); an arm therefore lasts 37.5 minutes.

7.4 Calibration

Table 5: Calibration results

Quantity	Measured value
Sustainable rate per t3.medium (p95 <= 800 ms, no errors)	16 req/s (6.4 concurrent requests)
Capacity sweep (rps: p95 ms, error rate)	4: 543, 0%; 8: 501, 0%; 12: 480, 0%; 16: 480, 0%; 20: 9048, 15%
ASG launch to healthy	28.5 s and 31.4 s from an empty group; 93.8 s on the very first launch of an earlier run
Lambda cold vs warm latency (median)	814 ms vs 610 ms (+205 ms); n = 24 cold, 48 warm, Lambda-served only
ALB weight-update propagation	12.1, 12.9, 11.8 s (three trials)

Two measurement problems were found and fixed during calibration, and are noted because they affect how far the numbers can be trusted. First, an initial capacity sweep showed errors at 12 requests per second that were really connection time-outs: the laptop's network address changed during the run and the load balancer's security group no longer matched it. A watchdog now keeps the security group pointed at the current address. Second, the first Lambda cold-start measurement did not wait for the load balancer's weight change to propagate (about 12 s), so cold and warm samples were mixed with EC2 responses; it was repeated with a 25 s wait and a filter that keeps only Lambda-served responses. The capacity figure comes from a coarse 4 requests-per-second grid: it is a lower bound between 16 and 20 requests per second.

7.5 What was not run live

AWS predictive scaling [3] was not run, because it needs 24 hours of metric history before it forecasts and each experiment arm lasts under 40 minutes. The seasonal strategy in the simulator is an emulation of that style of policy, not the AWS algorithm. Fargate was priced in the simulator only.

8. Evaluation

8.1 Set-up

Simulation results are for the 101 heavy functions, test days 13-14 scaled to 30 days, with the EC2 base and the measured capacity, unless stated. Costs are in US dollars at AWS ap-south-1 prices with Azure traffic. Test data were touched once for the final comparison; the quantile level, margin and all strategy parameters were chosen or fixed on validation. The sweeps over u for strategies 3 and 4 are reported in full rather than selecting one value.

8.2 Prediction accuracy

Accuracy is reported on test days, normalised by each function's training mean. Pinball loss is computed against the 60 per-minute values of each forecast hour; coverage is the share of minutes at or below the forecast.

Table 6: Pinball loss on test (normalised; lower is better)

q	LightGBM	Persistence	Rolling 6 h	Seasonal naive	Static train
0.5	0.231	0.211	0.248	0.277	0.323
0.7	0.244	0.246	0.274	0.302	0.388
0.8	0.228	0.246	0.264	0.287	0.387
0.9	0.186	0.210	0.218	0.241	0.340
0.95	0.145	0.172	0.177	0.198	0.274
0.99	0.058	0.105	0.089	0.126	0.142

Table 7: Coverage on test (ideal equals q)

q	LightGBM	Persistence	Rolling 6 h	Seasonal naive	Static train
0.5	0.573	0.614	0.607	0.614	0.647
0.7	0.737	0.730	0.727	0.716	0.769
0.8	0.832	0.792	0.791	0.769	0.832
0.9	0.915	0.859	0.862	0.825	0.888
0.95	0.958	0.897	0.908	0.858	0.921
0.99	0.990	0.935	0.958	0.895	0.953

Table 8: Mean absolute error of the forecast against the realised hourly quantile (test, normalised)

q	MAE LightGBM	MAE rolling 6 h
0.5	0.222	0.248
0.7	0.272	0.336
0.8	0.354	0.449

q	MAE LightGBM	MAE rolling 6 h
0.9	0.504	0.622
0.95	0.871	1.015
0.99	2.478	2.380

LightGBM has the lowest pinball loss at q = 0.7, 0.8, 0.9, 0.95, 0.99 and its coverage tracks the nominal level at the upper quantiles (for example 0.915 at q = 0.9 against 0.862 for rolling 6 h and 0.859 for persistence). At the median, persistence has the lower pinball loss (0.211 against 0.231), and LightGBM under-covers there (0.573). The 0.99 mean absolute error of LightGBM (2.48) is slightly worse than rolling 6 h (2.38), so the gain is not uniform. Figure 2 shows how the coverage curves compare.

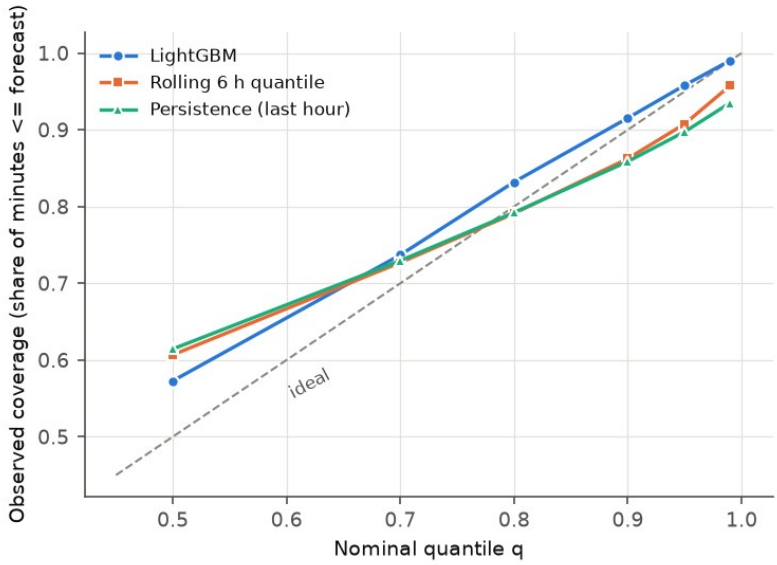


Figure 2: Observed coverage against nominal quantile on test days

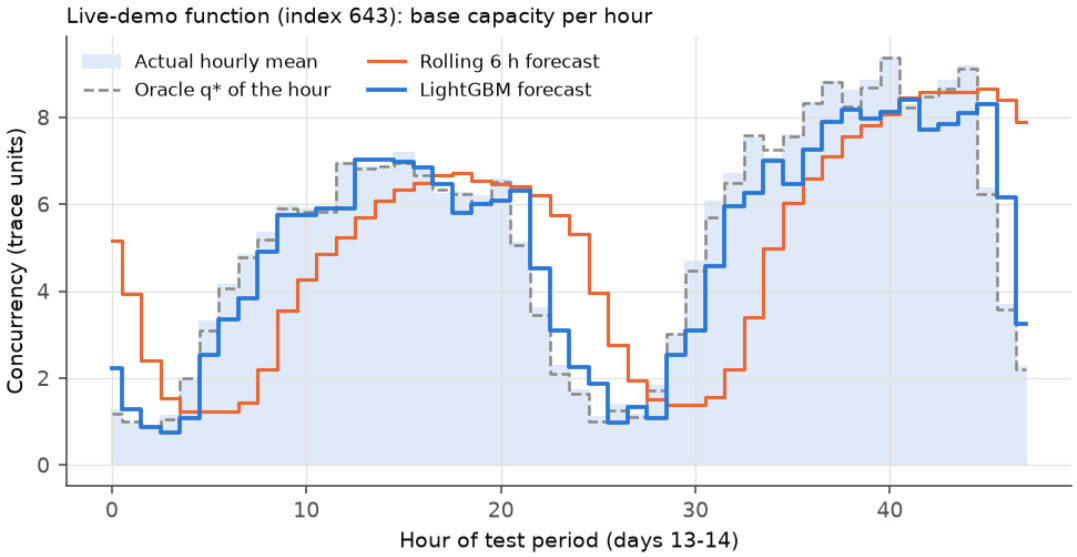


Figure 3: Hourly base capacity for the live-demo function: forecasts against the realised hourly quantile

8.3 Decision cost, utilisation and unserved load

Table 9: Test results, EC2 base (measured capacity)

Strategy	Cost / month	Unserved	Container util.	Lambda share	Energy kWh/mo
1 All-Lambda	$54,624	0.00%	n/a	1.00	1,040
2 Peak-sized x1.2	$873,014	0.49%	0.03	0.00	32,057
3 Reactive u=0.7	$37,485	9.21%	0.64	0.00	4,173
3 Reactive u=0.5	$52,479	7.93%	0.46	0.00	4,709
3 Reactive u=0.3	$87,464	6.82%	0.28	0.00	5,901
3 Reactive u=0.2	$131,197	6.27%	0.19	0.00	7,363
3 Reactive u=0.1	$262,393	5.68%	0.09	0.00	11,717
4 Seasonal u=0.7	$104,397	4.07%	0.24	0.00	6,549
4 Seasonal u=0.5	$146,156	2.04%	0.18	0.00	7,994
4 Seasonal u=0.3	$243,593	0.76%	0.11	0.00	11,255
5 Hybrid-static	$34,361	0.00%	0.86	0.29	3,239
6 Hybrid-rolling	$32,221	0.00%	0.91	0.22	3,455
7 Hybrid-ML	$32,140	0.00%	0.91	0.25	3,336

Strategy	Cost / month	Unserved	Container util.	Lambda share	Energy kWh/mo
8 Hybrid-oracle	$30,478	0.00%	0.94	0.18	3,567

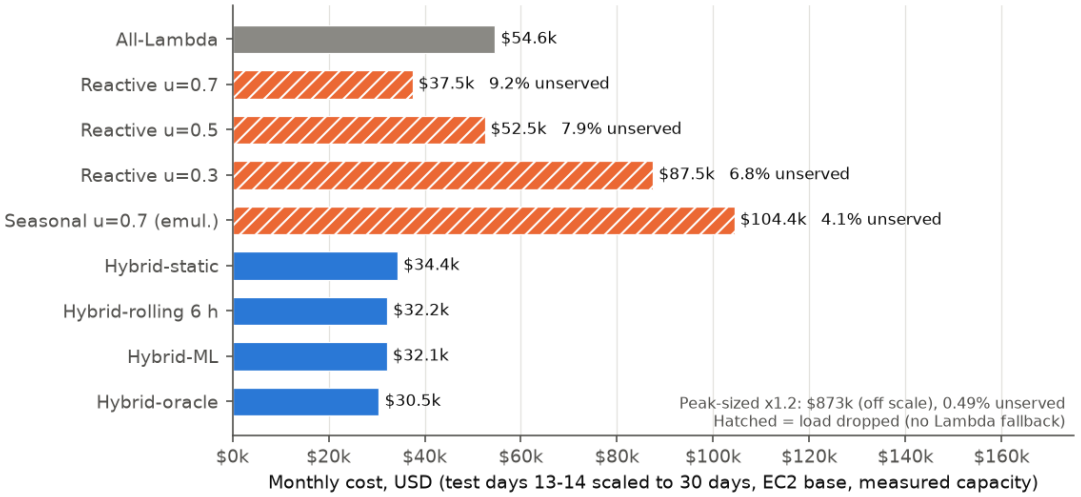


Figure 4: Monthly cost by strategy; hatching marks strategies that drop load

The hybrids are the only strategies with no unserved load and the lowest cost. Hybrid-rolling costs $32.2k per month, 41% below all-Lambda ($54.6k) and 14% below the cheapest reactive setting (u = 0.7, $37.5k), which also drops 9.2% of the load. Lowering u reduces unserved load only slowly and raises cost steeply: u = 0.3 costs $87.5k and still drops 6.8%, which is consistent with a controller that sizes capacity from load five minutes old and has no fallback for load that arrives in the meantime. The seasonal emulation drops less load than reactive but costs more at every u, because capacity set from the past maximum of the same hour is high all hour. Peak-sizing costs $873.0k and still drops 0.49% on test days because the test peak exceeded the training peak.

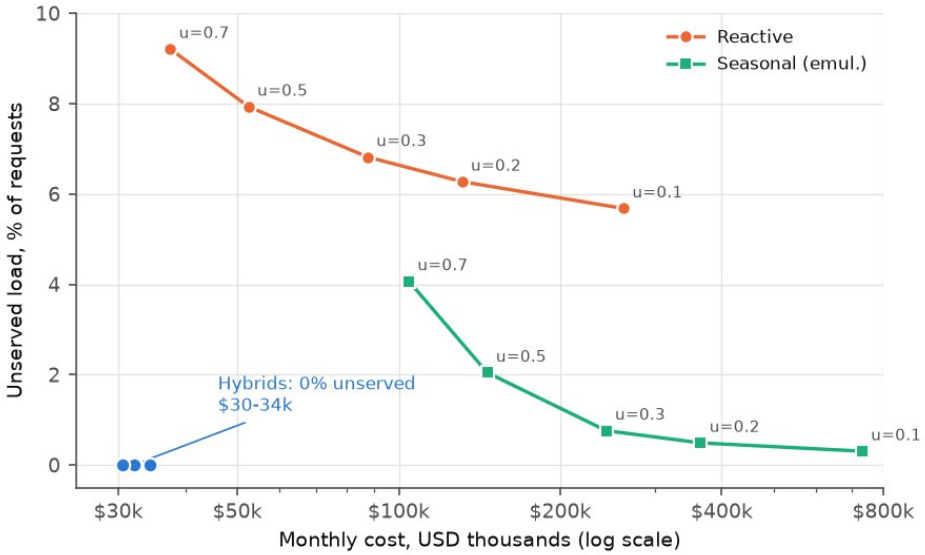


Figure 5: Cost against unserved load for the reactive and seasonal sweeps, and the hybrids

8.4 Does the ML forecaster help?

Table 10: Decision cost of the hybrid base policies

Hybrid variant	Validation cost / month	Test cost / month	Test Lambda share
Static quantile (5)	$34,777	$34,361	0.29
Rolling 6 h (6)	$33,979	$32,221	0.22
LightGBM (7)	$33,221	$32,140	0.25
Oracle (8)	$31,847	$30,478	0.18

The ML forecaster does not meaningfully beat the rolling 6-hour baseline. On validation LightGBM is 2.2% cheaper than rolling 6 h; on test it is 0.25% cheaper, a difference well inside what the sweeps over assumptions move. The whole gap from rolling 6 h to the hindsight oracle is only 5.4% of cost, which is the most any forecaster could gain on this population. Better accuracy at the upper quantiles shows up as a small gain that shrinks from 2.2% on validation to 0.25% on test, and on test the ML base sends a larger share of load to Lambda (0.25 against 0.22). Under the originally assumed capacity of 16 the same comparison gave -4.1% with 11.6% headroom, which is consistent with the preliminary exploration that suggested about 10% headroom. The practical conclusion is that the newsvendor rule, not the sophistication of the forecaster, delivers most of the benefit; a rolling quantile captures nearly all of it.

8.5 Latency

The simulator has no latency model. Latency is measured live (Section 8.8). The calibration gives the building blocks: an EC2 instance holds p95 near the service time up to its capacity and degrades sharply beyond it, and Lambda adds a cold-start overhead (Table 5).

8.6 Energy (estimate)

Energy is estimated with the Cloud Carbon Footprint methodology [9]: watts per vCPU = 0.74 + utilisation x (3.5 - 0.74), multiplied by vCPU-hours; Lambda uses vCPUs = memory in MB / 1792 at 50% utilisation. This is an **estimate**, not a measurement. On this basis all-Lambda uses the least energy (1,040 kWh per month), the hybrids use about 3,455 kWh, and the reactive strategies use 4,173 kWh or more. The reason is that the methodology charges an idle vCPU 0.74 W, so any strategy that keeps containers running pays for idle time that Lambda does not. The hybrid therefore saves money but, by this estimate, **not** energy relative to all-Lambda; it does use far less energy than reactive scaling at low u. The result is sensitive to the idle-power coefficient and to treating Lambda at 50% utilisation.

8.7 Per-function distribution and sensitivity

Across functions, the ratio of hybrid-rolling cost to the best single platform has median 0.86 (5th percentile 0.47, 95th percentile 1.01); hybrid-rolling is cheaper than all-Lambda for 82% of functions. For roughly a quarter of the functions (75th percentile ratio 1.00) the hybrid is no cheaper than the best single platform, so the saving comes from the other three quarters; we did not analyse which load characteristics separate the two groups.

Table 11: Sensitivity of cost to assumptions

Scenario (test, monthly)	All-Lambda	Reactive u=0.3	Hybrid-rolling	Hybrid-oracle
Baseline	$54,624	$87,464	$32,221	$30,478
Capacity per unit x0.5	$54,624	$174,929	$50,107	$49,606
Capacity per unit x1.5	$54,624	$58,310	$24,542	$22,415
Lambda price x0.5	$27,312	$87,464	$25,053	$24,803
Lambda price x2	$109,248	$87,464	$40,293	$36,108
EC2 price -30% (hypothetical)	$54,624	$61,225	$25,344	$23,256
Fargate base	$54,624	$81,014	$30,586	$28,761
Whole instances per function	-	-	$32,780	$31,202

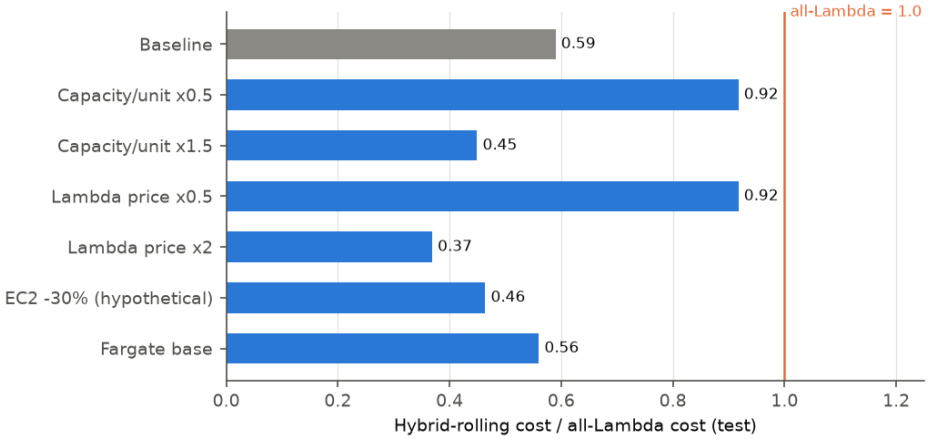


Figure 6: Hybrid-rolling cost relative to all-Lambda under each scenario

Table 12: Quantile-level sweep for hybrid-rolling on validation (offset 0 = theoretical q*)

Quantile level offset from q*	Validation cost / month
-0.2	$34,587
-0.1	$34,111
-0.05	$34,013
0	$33,979
0.05	$34,078
0.1	$34,422

The hybrid advantage is not unconditional. If a container unit carries half as much load as measured, hybrid-rolling saves only 8% against all-Lambda; if Lambda were half the price it would save 8%. With a Fargate base the savings are smaller ($30.6k against $32.2k) because Fargate capacity is priced higher per unit under the assumed 4-request task. The 30% EC2 discount is a hypothetical scenario, not an AWS-quoted figure. Under the assumed capacity of 16, hybrid-rolling cost $17.0k against $53.8k for all-Lambda (a larger saving), which shows how strongly the result depends on the measured capacity.

8.8 Live results and sim-to-real gap

Four arms were run one after another against the same trace window (function 643, scale factor 6.83, peak 28 req/s) with the same seed, from a laptop in Mumbai to the load balancer in ap-south-1. One trace hour lasts 375 s, so each arm is 37.5 minutes. Arm A is ASG target tracking on CPU at 50% with all traffic to EC2. Arms B and C use the controller of Section 6 with the static and the LightGBM forecast; arm D is arm C with the load multiplied by 4 during the fourth trace hour, after the forecast for that hour had been made. Predictive scaling was skipped (Section 7.5).

Table 13: Live results: latency, errors and routing

Arm	Requests	Errors	Lambda share	p50 ms	p95 ms	p99 ms	Over 800 ms or failed
A reactive (ASG target tracking)	23,511	0.1%	0.2%	445	566	809	1.2%
B hybrid-static	23,688	0.0%	0.7%	437	517	610	0.2%
C hybrid-ML	23,685	0.0%	0.5%	445	585	851	1.3%
D hybrid-ML + x4 spike	30,988	0.0%	20.0%	454	2618	3666	19.8%

Table 14: Live results: backend latency, resources, cost and energy estimate

Arm	EC2 p50 / p95 ms	Lambda p50 / p95 ms	Instance-hours	EC2 cost	Lambda cost	Total cost	Energy Wh (est.)
A reactive (ASG target tracking)	445 / 565	489 / 729	0.838	$0.038	$0.000	$0.038	3
B hybrid-static	437 / 516	459 / 760	1.863	$0.083	$0.002	$0.085	4
C hybrid-ML	444 / 584	480 / 752	0.827	$0.037	$0.002	$0.039	3
D hybrid-ML + x4 spike	454 / 2784	454 / 505	1.131	$0.051	$0.073	$0.124	5

Costs are computed from the instance-time recorded by the controller and from the CloudWatch Lambda duration and invocation counts at the Mumbai prices of Section 5.1; the load balancer, public addresses and CloudWatch are not included, because they are the same for every arm. Energy uses the estimate methodology of Section 8.6 with the measured mean CPU utilisation.

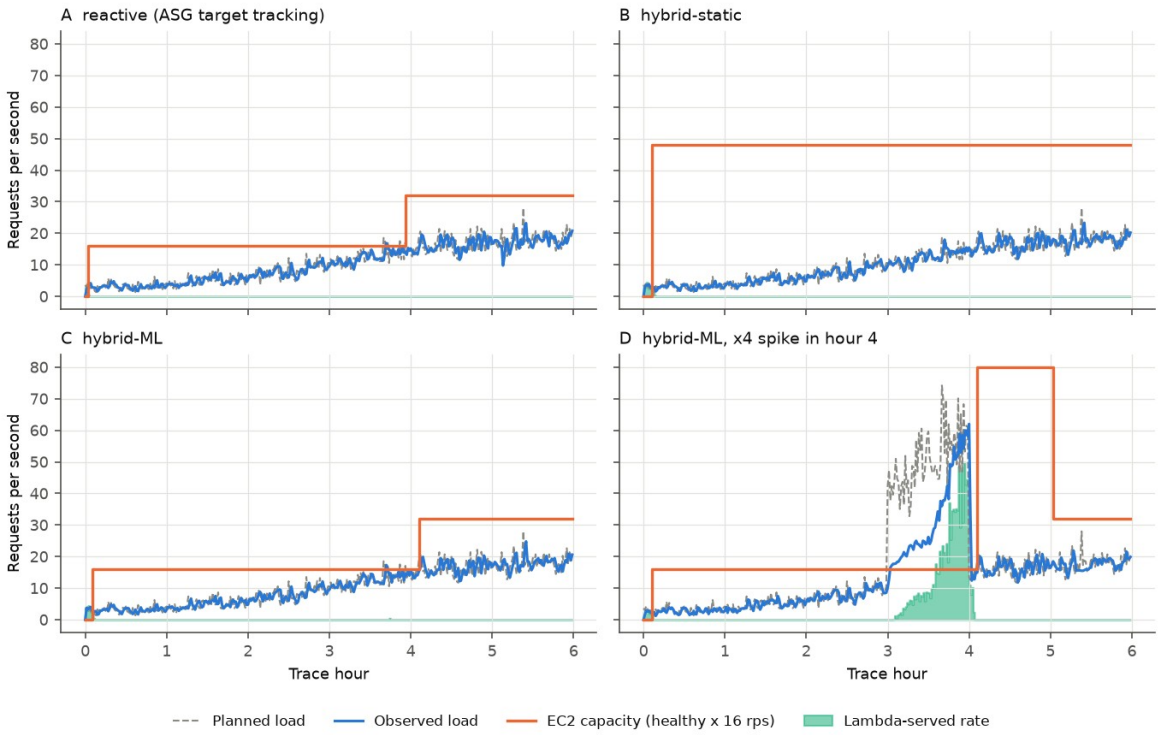


Figure 7: Live timelines: request rate, base capacity and EC2 share for each arm

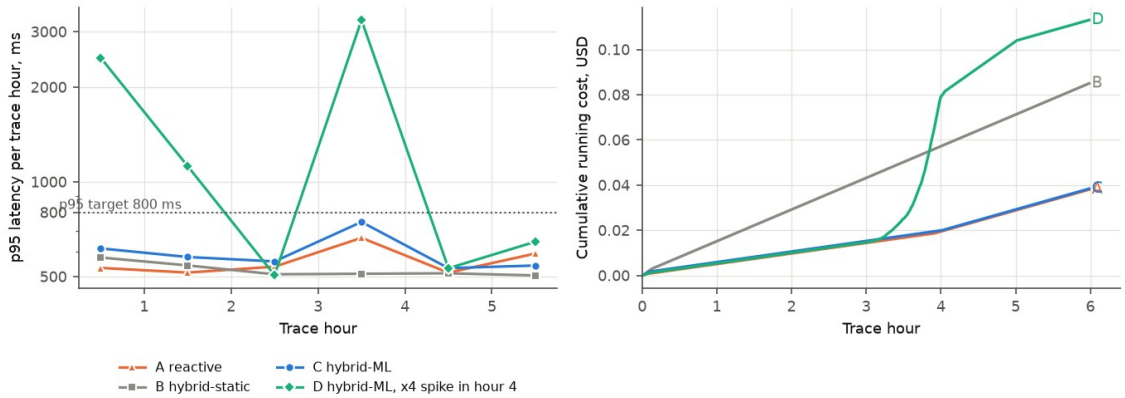


Figure 8: Live p95 latency per window and cumulative cost for each arm

Table 15: Reconciliation: live arms against a simulator replay of the same arm

Arm	Lambda share live	Lambda share replay	Gap (points)	Cost live	Cost replay	Cost gap
A reactive (ASG target tracking)	0.2%	0.0%	+0.2	$0.038	$0.038	+1%
B hybrid-static	0.7%	0.0%	+0.7	$0.085	$0.084	+1%
C hybrid-ML	0.5%	0.2%	+0.3	$0.039	$0.038	+2%
D hybrid-ML + x4 spike	20.0%	34.2%	-14.1	$0.124	$0.203	-39%

Reactive against hybrid (arms A and C). On this window the two are practically equal. Arm A had 31 failed requests out of 23,511 (0.13%), a p95 of 566 ms and a total cost of $0.0380; arm C had no failed requests, a p95 of 585 ms and $0.0386. Both used about 0.84 instance-hours. The window is a smooth ramp (from about 3 to 18 requests per second on average over 38 minutes, minute-level coefficient of variation 0.21), so CPU-based target tracking kept up. The simulator's dropped load for reactive scaling comes from minute-level bursts across the whole population of heavy functions, which this one window does not contain. The honest reading is that arms A to C show the hybrid works and costs about the same as reactive scaling here; they do **not** reproduce the simulated advantage.

Static against ML (arms B and C). Arm B cost $0.0853, 2.2 times arm C ($0.0386). The static base is the q* quantile of the training period, 45.5 requests per second, which is 3 instances for the whole run, while the window ramps up from a few requests per second; arm B therefore pays for idle capacity (mean CPU 11% against 30% in arm C). The forecast follows the ramp. This is the clearest live benefit of forecasting, and it is against a static base, not against the rolling baseline.

Unseen spike (arm D). In the spiked hour the planned load was 50.5 requests per second against a base of one instance (16 requests per second). No request failed in the whole run, and 48% of that hour's requests went to Lambda. But the hour's p95 latency was 3266 ms and the median latency of requests served by EC2 was 1946 ms, so 20% of all requests in arm D were slower than the 800 ms target or failed. Two causes are visible. First, the weights give EC2 its full measured capacity (about 16 requests per second), which leaves no headroom: with random arrivals a server fed at its limit builds a queue, so the p95-at-capacity measured in calibration does not hold on a sustained basis. Second, the load balancer's weight change takes about 12 s to apply and the observed rate is a 10-second average, so the first seconds of the spike went to EC2 unchanged. A safety factor on capacity (for example weights based on 70 to 80% of the measured capacity) would be the first fix; we did not test it. The Lambda-served requests in this arm were fast (p95 505 ms).

Delivered load. The load generator is closed-loop: each simulated user waits for its response before sending the next request, so when latency rises the offered rate falls. In the spiked hour the generator delivered 31.8 requests per second against 50.5 planned, and arm D served 30,988 requests in total against 37,928 planned. The spike was therefore smaller in practice than intended. The same effect makes the slow EC2 tail self-limiting, which means the latency numbers are, if anything, optimistic.

Sim-to-real gap. The replay assumes capacity is available instantly and that only the overflow reaches Lambda. For arms A, B and C the live Lambda share is +0.17 to +0.66 percentage points from the replay and the measured cost is +1.3% to +1.7% from it, which is small and, for arms B and C, in the direction of leakage (a little load reaches Lambda that the replay keeps on EC2). The replay of arm A uses the instance counts that the Auto Scaling Group actually had, so its cost agrees by construction. For arm D the gap is large: the live Lambda share is -14.1 points and the cost -39% from the replay, because the replay assumes the full planned spike is delivered and that exactly the overflow goes to Lambda, whereas live the generator delivered about 82% of the planned requests and EC2 absorbed more than its share. Weight changes take about 12 s to apply and new instances need 29 to 31 s to become healthy.

Start-up transient. In each arm the first trace hour shows a slow tail (for example arm D hour 0: EC2 p95 of about 2.4 s), when the first instance has only just become healthy and is serving its first requests.

8.9 Validation of the newsvendor rule

For every function and every test hour, a brute-force search over 401 candidate bases between zero and the hour's maximum was compared with the q*-quantile rule. Over all function-hours the rule's cost differs from the brute-force optimum by -0.0013% (a negative sign means the rule is marginally cheaper, because the grid is discrete). The preliminary exploration observed agreement within 0.5%; this check agrees to well under that. The check validates the rule against hindsight within an hour, not the forecast itself.

9. What happens when the prediction is wrong

The two directions of error behave differently, and the design makes the expensive direction mild.

- **Under-prediction (base too small).** Load above the base goes to Lambda, so in the simulation nothing is dropped and cost rises only by the Lambda price difference for the excess. The live spike arm (Section 8.8) showed the caveat: no request failed, but latency suffered while EC2 was fed up to its full capacity and the weights caught up. The architecture absorbs the load; it does not by itself protect latency.
- **Over-prediction (base too large).** Containers run idle. Cost rises by f per idle unit-minute until the next decision; no request is harmed. The asymmetry f versus L is what pushes q* above 0.5.
- **Slow reaction.** A purely reactive scaler is wrong in the damaging way: it under-provisions during a rise and has no fallback, so load is dropped or delayed.

Simulated unseen spike. In the test period one hour per function was multiplied by 3, 4 or 5 after the forecast for that hour had been made, with forecasts for later hours computed from the spiked history.

Table 16: Simulated spike results (monthly cost, test)

Spike	Hybrid-ML cost	Hybrid-rolling cost	Reactive u=0.7 cost	Reactive u=0.7 unserved	Hybrid unserved
none	$32,140	$32,221	$37,485	9.2%	0.0%
x3	$34,898	$34,387	$39,027	9.4%	0.0%
x4	$36,296	$35,492	$39,799	9.5%	0.0%
x5	$37,822	$36,599	$40,570	9.7%	0.0%

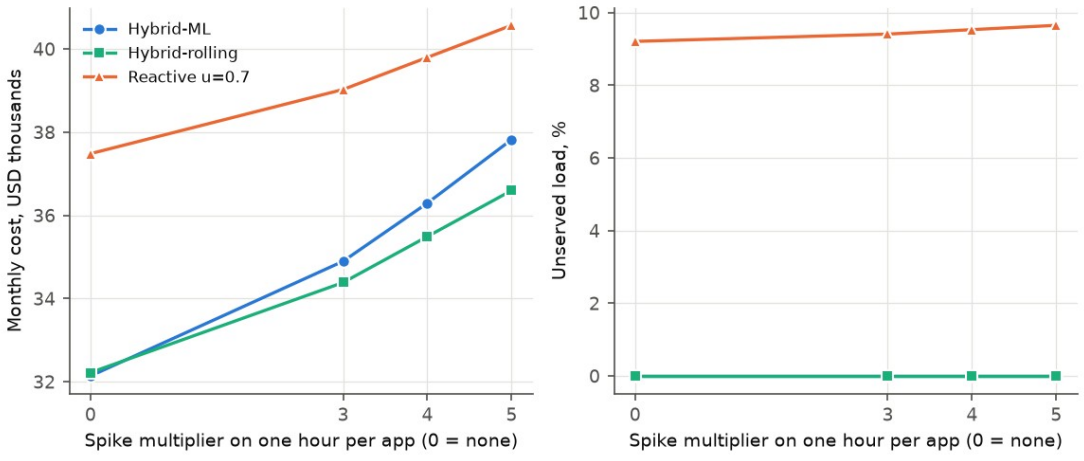


Figure 9: Effect of an unseen spike on cost and unserved load

The hybrids serve all load in every case; the cost of a x5 spike is $4.4k (rolling) to $5.7k (ML) per month above the no-spike case. The ML forecaster reacts worse than the rolling quantile after a spike (x5: $37.8k against $36.6k), because its features also include the spiked hour and it over-raises the next base. A production system would clip forecasts that follow a detected anomaly. The reactive controller is unaffected in kind but still drops about 9.5% of load in every case. Live arm D repeats this test on AWS (Section 8.8): there too nothing failed, but about 20% of requests exceeded the latency target, a cost the simulator does not model.

Remaining risks. The newsvendor argument assumes the forecast quantile is calibrated. Coverage at the median is not (Table 7), so a forecaster used at low q* (cheap containers relative to Lambda) would be miscalibrated where it matters most. A sustained level shift larger than the lookback window would be absorbed by Lambda for several hours at higher cost before the base catches up, and a fully new function with no history has no forecast at all and must start on Lambda.

10. Limitations

- **Azure traffic, AWS prices.** The trace was recorded on Azure Functions; running the same traffic on AWS with AWS prices is a modelling assumption, not an observation of an AWS customer.
- **Capacity assumptions.** Container capacity per unit is measured for one synthetic request type (400 ms service time, 20% CPU) on one instance type, from a coarse grid (16 req/s sustainable, 20 req/s failing). Real functions differ. Fargate capacity was never measured. Burstable t3 instances can run out of CPU credits under sustained load; credit balances were not measured in the short experiments.
- **Fractional capacity in the simulator.** Containers are billed as a shared pool; the whole-instance sensitivity row shows a small effect, but real fleets also have minimum sizes and availability-zone constraints.
- **Heavy-function filter.** Only 101 functions qualify; conclusions are about busy functions, not the median function, which is idle most of the day.
- **Forecaster scale.** The model trains on a small population (218k sampled rows); it is not tuned beyond a fixed configuration, and the benefit over rolling quantiles is within noise on test.
- **ALB leakage.** The load balancer splits by percentage, not overflow-only, and weight changes take about 12 s to propagate; the live system cannot match the simulator exactly (Section 8.8).
- **Short live runs.** Each arm is a time-compressed six-hour window run once, from one laptop and one function; no confidence intervals can be given. Cold-start behaviour of Lambda depends on how long environments stayed warm between arms. The window is a smooth ramp, so it does not stress reactive scaling the way the simulated population does.
- **Closed-loop load generator.** Locust users wait for each response, so when latency rises the offered load falls (arm D delivered about 63% of the planned rate in the spiked hour). Latency under overload is therefore understated.
- **No capacity headroom.** The controller gives EC2 its full measured capacity; the spike arm shows this leaves no margin. A safety factor was not tested.
- **Energy** is an estimate from a published methodology, not a measurement, and is sensitive to the idle-power coefficient.

- **Not covered.** Storage, networking, multi-region operation, security, and AWS predictive scaling (not run live) are outside this work.

11. Conclusion

CostScale keeps a cheap always-on base sized by a newsvendor quantile and sends the rest to Lambda. In simulation over 101 heavy Azure functions priced at AWS ap-south-1 rates with the container capacity measured on AWS, the hybrid with a rolling base costs $32.2k per month, 41% less than all-Lambda and 14% less than the cheapest reactive setting, while serving all load; reactive scaling drops between 5.7% and 9.2% of load at every utilisation target tried. The newsvendor rule matches brute-force search almost exactly.

Three results are weaker than the idea might suggest and are reported as found. The ML forecaster is only 0.25% cheaper than a rolling six-hour quantile on test, inside the noise of the assumptions, and the whole remaining headroom to the oracle is 5.4%. The advantage depends strongly on measured capacity per container and on the Lambda price (Section 8.7). By a documented estimate, the hybrid does not save energy relative to all-Lambda.

The live AWS runs confirm that the mechanism works: the controller sets instance counts from the forecast, shifts traffic to Lambda when load exceeds healthy capacity, and the replay of arms A to C agrees with the measurements to within about 2% in cost. They do not confirm a cost advantage over reactive scaling: on a smooth ramp arms A and C cost the same ($0.038 and $0.039), and the benefit shown was against a static base (arm B cost 2.2 times as much). The spike arm exposed a real weakness: with no failed requests, 20% of requests exceeded the latency target because EC2 was fed up to its measured capacity. Each live arm ran once, in compressed time, from one laptop, so the live evidence supports the design rather than a statistical ranking.

12. Future work

- **Storage tiering.** Extend the base-plus-overflow idea to storage classes once access-pattern data is available.
- **Network cost.** Include data-transfer and load-balancer charges, which this compute-only model omits.
- **Fargate base.** Measure Fargate capacity and run it live; the simulator prices it only under an assumed capacity.
- **Multi-region.** Place the base in the cheapest region and route overflow by latency and price.
- **Anomaly-aware forecasting.** Clip or down-weight history that follows a detected spike so the ML forecaster does not over-react (Section 9).
- **Overflow-only routing.** Replace weighted routing with per-request spillover, where each instance forwards a request to Lambda only when it is full, as Spandana [11] describes, to remove the ALB leakage measured in Section 8.8. We did not implement or test this.
- **Provider predictive scaling.** Run AWS predictive scaling [3] on a workload with 24 hours of history and compare directly.

References

[1] M. Shahrad, R. Fonseca, I. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich and R. Bianchini, "Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider," in Proc. 2020 USENIX Annual Technical Conference (USENIX ATC '20), pp. 205-218. https://www.usenix.org/conference/atc20/presentation/shahrad

[2] C. Zhang, M. Yu, W. Wang and F. Yan, "MArk: Exploiting Cloud Services for Cost-Effective, SLO-Aware Machine Learning Inference Serving," in Proc. 2019 USENIX Annual Technical Conference (USENIX ATC '19), pp. 1049-1062. https://www.usenix.org/conference/atc19/presentation/zhang-chengliang

[3] Amazon Web Services, "How predictive scaling works," Amazon EC2 Auto Scaling User Guide. https://docs.aws.amazon.com/autoscaling/ec2/userguide/predictive-scaling-policy-overview.html (accessed 24 October 2026).

[4] Amazon Web Services, "Configure Lambda function memory," AWS Lambda Developer Guide. https://docs.aws.amazon.com/lambda/latest/dg/configuration-memory.html (accessed 24 October 2026).

[5] Amazon Web Services, "Action types for listener rules" (forward actions and weighted target groups), Elastic Load Balancing User Guide. https://docs.aws.amazon.com/elasticloadbalancing/latest/application/rule-action-types.html (accessed 24 October 2026).

[6] "Newsvendor model," Wikipedia. https://en.wikipedia.org/wiki/Newsvendor_model (accessed 24 October 2026). The article cites F. Y. Edgeworth (1888) and K. J. Arrow, T. Harris and J. Marschak (1951) for the original results; those originals were not opened.

[7] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye and T.-Y. Liu, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," Advances in Neural Information Processing Systems 30 (NIPS 2017). https://papers.nips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html

[8] Microsoft Azure, "Azure Functions Trace 2019," Azure Public Dataset (CC-BY). https://github.com/Azure/AzurePublicDataset/blob/master/AzureFunctionsDataset2019.md (accessed 24 October 2026).

[9] Cloud Carbon Footprint, "Methodology." https://www.cloudcarbonfootprint.org/docs/methodology/ (accessed 24 October 2026).

[10] A. Raza, Z. Zhang, N. Akhtar, V. Isahagian and I. Matta, "LIBRA: An Economical Hybrid Approach for Cloud Applications with Strict SLAs," 9th IEEE International Conference on Cloud Engineering (IC2E), 2021; arXiv:2104.05491. https://arxiv.org/abs/2104.05491 (abstract read; accessed 24 October 2026).

[11] D. Dehigama, S. Jesalpura, Z. Xu, M. Nemeth, S. Zhu, M. Kogias and B. Grot, "Spandana: Reconciling Strict SLOs with Low Cost under Fine-Grained Load Fluctuations," arXiv:2606.30533, June 2026 (preprint). https://arxiv.org/abs/2606.30533 (abstract read; accessed 24 October 2026).

[12] D. Dehigama, S. Jesalpura, D. Schall, A. Katsarakis, M. Kogias, R. Kumar and B. Grot, "Flare: Leveraging Serverless Elasticity to Absorb Microservice Load Spikes," arXiv:2605.23707, May 2026 (preprint). https://arxiv.org/abs/2605.23707 (abstract read; accessed 24 October 2026).

Appendix A: Reproduction steps

All commands are run from the repository root. Credentials are read from `.env` and mapped to standard AWS variable names by `infra/env.sh`; they are never printed or committed.

```
python3 -m venv .venv && .venv/bin/pip install numpy pandas pyarrow lightgbm
scikit-learn matplotlib boto3 locust python-docx python-pptx
tar -xJf azurefunctions_dataset2019_*.tar.xz -C data/raw        # about 2.2 GB
make data         # two-pass pipeline -> results/dataset_summary.json
make sim          # simulator -> results/sim_results.json
make forecast     # LightGBM + baselines + spike test ->
results/forecast_results.json
. infra/env.sh && .venv/bin/python infra/budgets.py <email>    # budget alerts
make up           # create the stack (ASG at 0 instances; ALB bills about
$0.024/h)
make calibrate    # -> results/calibration.json ; then re-run make sim and make
forecast
make run          # arms A-D, about 2.8 h, -> results/live/
make down         # destroy everything and verify with the tagging API
.venv/bin/python controller/analyze.py ; make report ; make ppt
```

A.1 Parameters

Table 17: Key parameters

Parameter	Value
Region	ap-south-1
Split	days 1-10 train, 11-12 validation, 13-14 test
Quantiles	0.5, 0.7, 0.8, 0.9, 0.95, 0.99
LightGBM	300 trees, learning rate 0.05, 15 leaves, min 50 samples per leaf, subsample 0.8, seed 0
Lambda	python3.12, 1769 MB, 30 s timeout
Instance	t3.medium, AL2023, ASG 0-8, ALB health check every 5 s
Live request	400 ms total, 80 ms CPU